

How to Use This Coding Guide

The assessment includes Python programming questions (candidate reports mention ~2 coding problems).

These analyst/data-science flavoured problems emphasise data manipulation, counting and aggregation.

Every Python solution has been executed and verified against its sample output.

Coding Problems

Problem 1: Most Frequent Word

Difficulty: Easy | Pattern: Hash Map / Counting

Problem Statement

Given a sentence, return the word that appears most frequently. If several words tie, return the one that appears first. Comparison is case-insensitive.

Input Format

A single line: the sentence.

Output Format

The most frequent word (lowercase).

Constraints

Words are separated by single spaces.

Sample Input

```
the cat sat on the mat the cat
```

Sample Output

```
the
```

Python Solution

```
def most_frequent_word(sentence):
    counts = {}
    best_word, best_count = None, 0
    for word in sentence.lower().split():
        counts[word] = counts.get(word, 0) + 1
        if counts[word] > best_count:
            best_count = counts[word]
            best_word = word
    return best_word

print(most_frequent_word('the cat sat on the mat the cat'))
```

Step-by-Step Explanation

Count each word in a dictionary. Track the best word as you go; updating only when a strictly higher count appears preserves the first word on ties. 'the' occurs 3 times.

Dry Run

```
the->1 (best=the)
cat->1
sat->1
on->1
the->2 (best=the)
mat->1
the->3 (best=the)
cat->2
Result = the
```

Time Complexity: **O(N)** | Space Complexity: **O(K) distinct words**

Problem 2: Find the Missing Number

Difficulty: *Easy* | Pattern: *Math / Summation*

Problem Statement

An array contains n distinct numbers taken from the range 0 to n . Exactly one number in that range is missing. Return the missing number.

Input Format

First line: n . Second line: the n numbers.

Output Format

The missing number.

Constraints

$1 \leq n \leq 10^6$

Sample Input

```
8
3 0 1 5 2 7 6 8
```

Sample Output

```
4
```

Python Solution

```
def missing_number(nums):
    n = len(nums)
    expected = n * (n + 1) // 2
    return expected - sum(nums)

print(missing_number([3, 0, 1, 5, 2, 7, 6, 8]))
```

Step-by-Step Explanation

The sum of $0..n$ is $n(n+1)/2$. Subtracting the actual sum of the array yields the missing value. Here $n=8$, expected sum = 36, actual sum = 32, missing = 4.

Dry Run

$n=8$, expected= $8*9/2=36$

```
sum(array)=32
36-32=4
Result = 4
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 3: FizzBuzz

Difficulty: *Easy* | Pattern: *Simulation / Modulo*

Problem Statement

Print numbers 1..n, but for multiples of 3 print 'Fizz', for multiples of 5 print 'Buzz', and for multiples of both print 'FizzBuzz'. Output values space-separated.

Input Format

A single integer n.

Output Format

The FizzBuzz sequence on one line, space-separated.

Constraints

$1 \leq n \leq 10^4$

Sample Input

15

Sample Output

1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz

Python Solution

```
def fizzbuzz(n):
    out = []
    for i in range(1, n + 1):
        if i % 15 == 0:
            out.append('FizzBuzz')
        elif i % 3 == 0:
            out.append('Fizz')
        elif i % 5 == 0:
            out.append('Buzz')
        else:
            out.append(str(i))
    return ' '.join(out)

print(fizzbuzz(15))
```

Step-by-Step Explanation

Check divisibility by 15 first (both 3 and 5), then 3, then 5, otherwise use the number itself. Checking 15 first avoids printing Fizz/Buzz separately for common multiples.

Dry Run

```
3->Fizz, 5->Buzz, 15->FizzBuzz, others->number
Result = 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 4: Two Sum

Difficulty: *Easy-Medium* | Pattern: *Hash Map*

Problem Statement

Given an array of integers and a target, return the 0-based indices of the two numbers that add up to the target. Exactly one solution exists.

Input Format

Line 1: the array. Line 2: the target.

Output Format

The two indices, space-separated, in increasing order.

Constraints

$2 \leq \text{length} \leq 10^5$

Sample Input

```
2 7 11 15
9
```

Sample Output

```
0 1
```

Python Solution

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return (seen[need], i)
        seen[x] = i
    return None

i, j = two_sum([2, 7, 11, 15], 9)
print(i, j)
```

Step-by-Step Explanation

For each number, compute the complement needed to reach the target. If the complement was already seen, we have the pair. Otherwise store the current number's index. $2 + 7 = 9$ gives indices 0 and 1.

Dry Run

```
i=0 x=2 need=7 not seen; store 2->0
i=1 x=7 need=2 seen at 0 -> return (0,1)
Result = 0 1
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 5: Find Duplicates in a List

Difficulty: *Easy* | Pattern: *Set / Counting*

Problem Statement

Given a list of integers, return all values that appear more than once, in ascending order, space-separated. If there are none, print 'none'.

Input Format

First line: n. Second line: the n integers.

Output Format

The duplicate values in ascending order, or 'none'.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
8
4 3 2 7 8 2 3 1
```

Sample Output

```
2 3
```

Python Solution

```
def find_duplicates(nums):
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    dups = sorted(v for v, c in counts.items() if c > 1)
    return ' '.join(map(str, dups)) if dups else 'none'

print(find_duplicates([4, 3, 2, 7, 8, 2, 3, 1]))
```

Step-by-Step Explanation

Count occurrences in a dictionary, then keep values whose count exceeds one and sort them. 2 and 3 each appear twice.

Dry Run

```
counts: 2->2, 3->2, others->1
dups=[2,3] sorted
Result = 2 3
```

Time Complexity: **O(N log N) due to sort** | Space Complexity: **O(N)**

Problem 6: Reverse the Words in a Sentence

Difficulty: *Easy* | Pattern: *String Manipulation*

Problem Statement

Given a sentence, reverse the order of the words while keeping each word intact. Collapse extra spaces.

Input Format

A single line: the sentence.

Output Format

The sentence with word order reversed.

Constraints

$1 \leq \text{length} \leq 10^4$

Sample Input

```
data science is fun
```

Sample Output

```
fun is science data
```

Python Solution

```
def reverse_words(sentence):
    words = sentence.split()
    return ' '.join(reversed(words))

print(reverse_words('data science is fun'))
```

Step-by-Step Explanation

`split()` breaks on whitespace and drops empty tokens; `reversed()` flips the order; `join()` rebuilds the string. The words are emitted last-to-first.

Dry Run

```
split -> ['data', 'science', 'is', 'fun']
reversed -> ['fun', 'is', 'science', 'data']
join -> 'fun is science data'
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 7: Second Largest Unique Value

Difficulty: **Easy-Medium** | Pattern: **Set + Sorting**

Problem Statement

Given a list of integers, return the second largest distinct value. If it does not exist, print -1.

Input Format

First line: n. Second line: the n integers.

Output Format

The second largest distinct value, or -1.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
6
10 10 9 8 8 7
```

Sample Output

```
9
```

Python Solution

```
def second_largest_unique(nums):
    distinct = sorted(set(nums), reverse=True)
```

```
        return distinct[1] if len(distinct) >= 2 else -1

print(second_largest_unique([10, 10, 9, 8, 8, 7]))
```

Step-by-Step Explanation

Remove duplicates with a set, sort in descending order, and take the element at index 1. Distinct sorted values are [10, 9, 8, 7]; the second is 9.

Dry Run

```
set -> {7,8,9,10}
sorted desc -> [10,9,8,7]
index 1 -> 9
```

Time Complexity: **O(N log N)** | Space Complexity: **O(N)**

Problem 8: Group Frequencies (Value Counts)

Difficulty: **Medium** | Pattern: **Hash Map / Aggregation**

Problem Statement

Given a list of category labels, output each distinct label with its count, sorted by count descending and then label ascending. This mirrors a pandas `value_counts()`.

Input Format

First line: `n`. Second line: `n` space-separated labels.

Output Format

One 'label:count' pair per line.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
7
apple banana apple cherry banana apple cherry
```

Sample Output

```
apple:3
banana:2
cherry:2
```

Python Solution

```
def value_counts(labels):
    counts = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    ordered = sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return '\n'.join(f'{k}:{v}' for k, v in ordered)

data = ['apple', 'banana', 'apple', 'cherry',
        'banana', 'apple', 'cherry']
print(value_counts(data))
```

Step-by-Step Explanation

Count each label, then sort by (-count, label) so higher counts come first and ties break alphabetically. This is the logic behind a frequency table used constantly in analytics.

Dry Run

```
counts: apple=3, banana=2, cherry=2  
sort key (-count, label): apple(-3), banana(-2,b), cherry(-2,c)  
Result lines: apple:3 / banana:2 / cherry:2
```

Time Complexity: **$O(N \log N)$** | Space Complexity: **$O(K)$**

Tredence

Python Coding Problems

Applied Role(s):

Analyst - Data Scientist

Online Assessment Preparation Material

Version 1.0 • Last Updated: July 2026

Compiled from verified previous-year patterns, community-reported questions and research-backed company-style practice.

Table of Contents

How to Use This Coding Guide	1
Coding Problems	1